

Document Ingestion and Structural Extraction for RAG

Introduction

Before any chunk can be embedded, indexed, or retrieved, the source document it comes from must first be converted from its original file format into a sequence of structural units that the rest of the ingestion pipeline can reason about, a process that determines, upstream of chunking itself, which headings are recognized, which tables are recognized as tables rather than flat paragraphs, and which hierarchy of sections a given piece of content belongs to.

This document describes how source documents are loaded and structurally parsed, the specific capabilities and limitations of each supported source format, how section hierarchy is tracked during extraction, how tables are serialized into a canonical form before chunking, and the operational considerations around re-ingesting an updated version of a previously ingested document.

Supported Source Formats

This system supports two source formats for structured ingestion: Markdown files, loaded and parsed as plain text using their native heading and table syntax, and Word documents in the DOCX format, loaded using python-docx and converted internally into the same canonical structural representation that Markdown parsing produces, so that downstream chunking logic operates on a single unified representation regardless of which format a document originally arrived in.

PDF files are also accepted as an input format, but PDF ingestion has a materially different, and more limited, structural fidelity than Markdown or DOCX ingestion, because PDF text extraction returns a flat stream of text without any reliable structural markup indicating headings, tables, or even paragraph boundaries in many cases, which means a PDF's original visual structure, including any tables it contains, is generally not recovered as structured data at all during ingestion.

Choosing to support only two formats for full structural extraction, rather than attempting best-effort structural recovery from PDFs as well, reflects a deliberate scope decision: Markdown and DOCX both carry genuine, unambiguous structural markup, whether through heading syntax or paragraph style metadata, that can be parsed reliably, whereas any structural recovery attempted from raw PDF text would necessarily be a heuristic guess rather than a parse of real, author-provided structure.

Structural Unit Extraction

Every loaded document is converted into an ordered sequence of structural units of four kinds: headings, paragraphs, list items, and tables, processed top to bottom through the document's text, with a heading always recognized by Markdown's `#`-style syntax regardless of whether the original source was a Markdown file directly or a DOCX file whose own heading styles were translated into that same syntax during loading.

A paragraph is recognized as a block of one or more non-blank lines not matching any heading, list, or table pattern, with a blank line marking the end of the current paragraph and the start of a new structural unit; this means paragraph boundaries in the source document, specifically where an author chose to insert a blank line, directly determine where one paragraph-level structural unit ends and the next begins, which is the mechanism by which later semantic grouping can only ever place a chunk boundary between two already-distinct authored paragraphs, never partway through the middle of one.

List items, recognized by a leading bullet marker or a numbered-list marker, are each extracted as their own individual structural unit rather than being merged together into one combined list block, which means a list of

several bullet points becomes several separate structural units in sequence, each eligible to be independently grouped or split during the later semantic chunking pass just like an ordinary paragraph would be. Table detection specifically looks for genuine GitHub-Flavored-Markdown pipe-table syntax, meaning a header row immediately followed by a dash-and-pipe separator row, which is a well-defined, unambiguous text pattern rather than a fabricated or heuristic capability, and this detection logic is applied uniformly regardless of whether that pipe-table syntax originated directly from a hand-authored Markdown file or was generated by the DOCX loader translating a genuine, structured Word table into the same pipe-table representation.

Section Hierarchy Construction

Section hierarchy is tracked using a stack of currently open headings, where encountering a new heading at a given level closes out, meaning pops off the stack, every previously open heading at that same level or any deeper level, before the new heading is pushed onto the stack itself, which ensures the stack always reflects the correct current nesting path regardless of how deeply headings are nested or how many sibling sections at the same level precede the current one.

Every structural unit extracted while a given set of headings is open on the stack is tagged with a section path built by joining the titles of every currently open heading from the outermost to the innermost, which is what allows a chunk, much later in the pipeline, to retain a record of exactly which hierarchical position in the source document it was extracted from.

A document whose heading levels skip unexpectedly, for example jumping directly from a top-level heading to a third-level heading with no second-level heading in between, is still parsed consistently under this stack-based rule, since the rule only cares about relative heading depth at each step, not about whether every intermediate depth was actually used by the document's author, though a reader relying on section path metadata to understand document structure may find such a skipped hierarchy less intuitive to interpret than a document with strictly consecutive heading levels.

Heading Style Translation Between DOCX and Markdown

A DOCX file encodes headings through named paragraph styles, such as a paragraph explicitly styled as "Heading 1" or "Heading 2" in Word's own style system, rather than through any inline text syntax, so the DOCX loader must map each paragraph's style name to the corresponding heading level before that paragraph can be treated identically to a Markdown ``#``-prefixed heading line by the rest of the pipeline. Native DOCX heading styles, meaning Word's own built-in "Heading 1" through "Heading 6" style names, are recognized directly and map predictably to their corresponding numeric heading level, which is the common and reliable case for documents authored directly in Word using Word's standard style gallery rather than manually formatted to merely look like a heading without using the actual heading style.

A paragraph that is manually formatted to visually resemble a heading, for example simply made bold and given a larger font size without actually being assigned a heading paragraph style, is not recognized as a heading by the DOCX loader, since the loader inspects the paragraph's assigned style rather than its visual appearance, which means such a paragraph is ingested as an ordinary paragraph structural unit, and any content that would otherwise have been nested under it structurally is instead treated as belonging to whatever heading was most recently and genuinely styled as one.

Table Serialization

Once a table is detected, whether parsed directly from Markdown pipe-table syntax or converted from a genuine DOCX table object, it is serialized into one canonical Markdown representation, consisting of a header row, a separator row, and one row per data row, each row expressed as pipe-delimited cell values, which ensures that two tables carrying identical information produce identical structural unit content regardless of which source format they originally came from.

This canonical serialization happens before the table-fragmentation and chunking logic ever examines the table, so by the time chunking decides whether a table needs to be split into multiple fragments to respect its token budget, it is always working from this same uniform representation rather than needing separate handling logic for Markdown-sourced tables versus DOCX-sourced tables.

Because serialization always re-renders a table from its parsed structural data rather than preserving the original source formatting verbatim, minor original whitespace or alignment differences in how a table was originally typed, whether in Markdown or in Word, do not propagate into the final chunk content, which keeps

the eventual embedded representation of a table consistent and predictable rather than sensitive to incidental formatting choices made by whoever originally authored the document.

Versioning and Logical Document Identity

Every ingested document is assigned a logical document identifier that remains stable across subsequent re-ingestions of an updated version of that same source document, which allows the system to recognize that a newly ingested version supersedes a previous one logically, rather than treating the new version as an entirely unrelated document that happens to share similar content.

When a document is re-ingested under the same logical document identifier, the newly produced chunks become the current version, and queries against that logical document identifier are served from the current version going forward, while the previous version's chunks are retained for traceability rather than deleted outright, which supports being able to answer, after the fact, exactly what content a query would have retrieved against an earlier version of a document.

This versioning design means a single logical document can have multiple physical chunk sets stored over time, only one of which is marked current at any given moment, and retrieval queries in normal operation are always scoped to current documents only, so an older, superseded version's chunks do not compete for ranking against the current version's chunks during ordinary vector search.

Format Fidelity Comparison

Source format	Heading detection	Table detection	Known limitation
Markdown (.md)	Native `#` syntax, reliable	GFM pipe tables, reliable	Depends on the author u
DOCX (.docx)	Native paragraph heading styles, reliable	Structured table objects, re-serialized to the same pipe syntax	Manually-formatted pseu
PDF (.pdf)	Not detected; flat text only	Not detected; flat text only	Layout and tabular struc

Failure Modes

A malformed table, for example a Markdown table missing its required dash-and-pipe separator row immediately after the header row, is not recognized as a table at all by the detection logic, and is instead ingested as a sequence of ordinary paragraph lines, each pipe-delimited row treated as flat prose text rather than as structured tabular data, which silently degrades that table's retrievability as tabular evidence without producing any explicit ingestion error.

An orphaned heading, meaning a heading unit with no body content directly following it before either the document ends or the next heading begins, is still extracted and retained as a valid structural unit on its own, but it carries very little independent retrieval value once it eventually becomes, or contributes to, a chunk, since a heading's text alone is typically too short and too generic to serve as useful evidence for a specific query. PDF text garbling is a known and accepted limitation rather than a bug to be fixed within the current scope: because PDF text extraction has no access to the original document's layout information, content originally laid out in multiple columns can be extracted with its reading order scrambled, interleaving text from two unrelated columns into a single run of extracted text, which can produce ingested content that does not correspond to any coherent original sentence at all.

Encoding, Locale, and Special Character Handling

Markdown files are read as UTF-8 encoded text, which reliably handles the overwhelming majority of real-world documents, including documents that mix English prose with occasional non-English terms, accented characters, or symbols, but a source file saved under a different encoding and never converted to UTF-8 before ingestion can produce garbled or entirely unreadable characters in its extracted content, a failure that occurs at the file-loading step and before any structural parsing has a chance to run.

Typographic characters such as curly quotation marks, em dashes, and non-breaking spaces, which word processors commonly substitute automatically for their plain ASCII equivalents, are preserved as-is through structural extraction rather than being normalized to a plain-text equivalent, which means two otherwise identical sentences, one typed with straight quotes and one with curly quotes substituted automatically by an editor, are not treated as identical strings at the structural extraction level, though this distinction generally does not affect embedding-based retrieval, which operates on semantic meaning rather than exact character-level string matching.

DOCX files converted from other tools, or containing content originally pasted from a web page, occasionally carry embedded formatting artifacts or unusual whitespace characters that are not visually apparent when the document is opened in Word but can appear in the raw paragraph text python-docx extracts, which is a source of occasional minor noise in extracted paragraph content that structural extraction does not attempt to detect or clean beyond the ordinary whitespace handling applied to every paragraph.

Idempotency and Duplicate Ingestion Prevention

Re-running an ingestion job against the exact same source file is a scenario that needs to be handled deliberately rather than left to produce an unbounded number of duplicate document versions, since an ingestion job triggered more than once against unchanged content, for example due to a retried request after a transient failure, should not silently double the number of chunks representing that same unchanged content in the corpus.

Duplicate ingestion prevention is handled at the document level by comparing a content hash of the newly loaded source file against the content hash recorded for the current version of that logical document, and when the two hashes match exactly, the ingestion job treats the request as already satisfied and does not produce a new version at all, rather than re-running structural extraction and chunking against content that is byte-for-byte identical to what is already the current version.

This hash-based check only prevents exact duplicate re-ingestion of unchanged content; it does not detect or deduplicate near-duplicate content, such as two different source documents that happen to describe the same underlying concept in different words, since that kind of overlap is a matter of semantic similarity rather than exact file content and is therefore outside the scope of what the ingestion-time duplicate check is designed to catch.

Minimum Viable Document Requirements

A document containing no recognizable headings at all is still accepted and ingested successfully rather than rejected, in which case every structural unit in the document is treated as belonging to a single flat, top-level section with an empty section path, since the absence of headings is a valid, if unstructured, document shape rather than an ingestion error condition.

A document that is empty, or that contains only whitespace once loaded, is rejected outright at ingestion time rather than being allowed to proceed through structural extraction and chunking, since there is no structural unit of any kind for the rest of the pipeline to meaningfully operate on, and allowing such a document to proceed would simply produce a document record with zero associated chunks.

A document consisting of a single very large table and no other content ingests successfully and produces whatever number of table fragments the table-fragmentation logic determines are needed to respect the table token budget, with no prose chunks at all accompanying it, which is a valid, if unusual, outcome rather than a condition requiring any special handling distinct from the ordinary table-fragmentation path already described in this document.

Limitations

Structural extraction in this system is deliberately syntax-based rather than inference-based: it recognizes real, unambiguous structural markup, genuine heading styles, genuine pipe-table syntax, genuine list markers, and makes no attempt to infer structure from visual formatting, layout position, or statistical text patterns, which keeps its behavior predictable and auditable but means any source document that conveys its structure purely through visual appearance rather than through this recognized markup will not have that structure recovered automatically during ingestion.

Re-ingestion and versioning in this system operate at the level of an entire logical document; there is no mechanism to re-ingest and supersede only a single section or a single chunk within an already-ingested document, so even a small, localized edit to one part of a large source document requires the entire document to be re-ingested and re-chunked as a new current version.